

Computability: Recitation 4

April 6th, 2025

1 Turing machines: motivation

DFAs have limitations: they must read the input exactly once, and they do not have an external memory (beside their current state) to write to during their run. In contrast, a Turing machine (TM) is a more powerful computational model, equipped with infinite external memory and the ability to read and write freely.

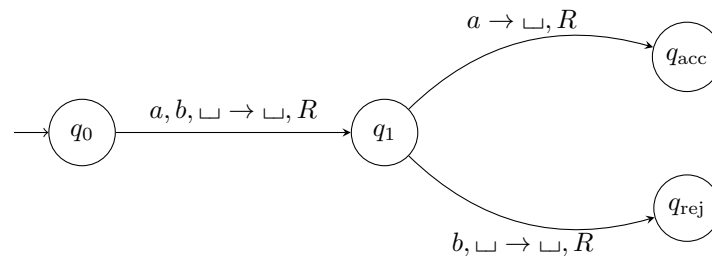
The memory is a *tape* of cells that stretches infinitely in both directions. Each cell contains a letter. Initially the input is written on the tape, and every other cell contains a default blank symbol, denoted $\sqcup$. The machine's head initially points to the first letter of the input.

Similarly to a DFA, a TM has internal states and transitions that describe how it works. There are a couple of differences:

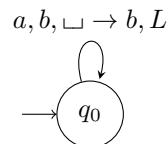
- A transition has additional information: which letter to write to the tape, and whether to move the head left or right.
- The machine halts by moving into a *final state*. For decision problems, where we want a binary output, we can define the final states to contain an accepting state and a rejecting state.

2 Examples

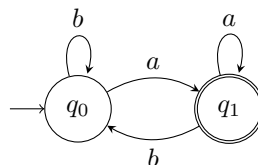
Example: The following TM checks whether the second letter of the input is a , and accepts or rejects accordingly. It also deletes (writes $\sqcup$) the first two letters of the input.



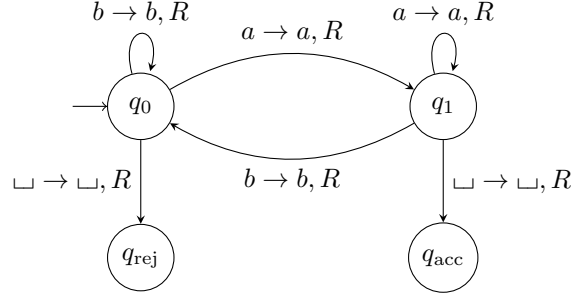
Example: The following TM does not halt. It fills the tape to the left of the input with b 's. It also overrides the first letter of the input with b .



Example: The following DFA $\mathcal{A}$ recognizes the language $L = \{w \in \{a, b\}^* : w \text{ ends in } a\}$.



The following TM behaves like $\mathcal{A}$: it reads the input from left to right, and when it encounters the end of the input (the $\sqcup$ symbol), it jumps to q_{acc} or q_{rej} , depending on whether it should accept or reject. The same approach works for any DFA.



3 Formal definition

Definition 1 (Turing Machine). A TM contains the following:

$$M = \langle \Sigma, \Gamma, \sqcup, Q, q_0, F, \delta \rangle$$

- A finite input alphabet Σ .
- A finite tape alphabet Γ , such that $\Sigma \subseteq \Gamma$.
- A blank symbol $\sqcup \in \Gamma \setminus \Sigma$.
- A finite non-empty set of states, Q .
- An initial state $q_0 \in Q$.
- A set of final states, $F \subseteq Q$.
- A transition function δ .

The transition function takes a non-final state and a tape letter, and outputs: the next state, a tape letter to write, and whether to move left or right. Formally,

$$\delta : (Q \setminus F) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

where L and R stand for left and right movements.

Definition 2 (Configuration). A configuration is composed of the contents of the tape, the current state, and the position of the head.

If the tape contains $x = x_1 x_2 \cdots x_n$, the automaton is in state q , and the head is positioned over x_i , we represent the configuration as

$$x_1 \cdots x_{i-1} q x_i x_{i+1} \cdots x_n.$$

Definition 3 (Run). A run of a TM M on an input word $w \in \Sigma^*$ is a finite or infinite sequence of configurations $c_1, c_2, \dots, c_T$ (if the run is infinite we denote $T = \infty$) such that:

1. The first configuration is

$$c_1 = q_0 w_1 w_2 \cdots w_n,$$

where $w = w_1 w_2 \cdots w_n$ and the head is positioned at the leftmost symbol.

2. For all $0 \leq j < T$, if

$$c_j = x_1 \cdots x_{i-1} q x_i x_{i+1} \cdots x_n,$$

then the next configuration c_{j+1} is the following configuration as determined by the transition function δ ,

- If $\delta(q, x_i) = (q', y_i, L)$, then

$$c_{j+1} = x_1 \cdots x_{i-2} q' x_{i-1} y_i x_{i+1} \cdots x_n.$$

- If $\delta(q, x_i) = (q', y_i, R)$, then

$$c_{j+1} = x_1 \cdots x_{i-1} y_i q' x_{i+1} \cdots x_n.$$

Definition 4 (Decision TM). A Turing machine for a decision problem is a machine with final states

$$F = \{q_{acc}, q_{rej}\},$$

where reaching q_{acc} means the machine accepts, and reaching q_{rej} means it rejects.

We say that a decision TM M accepts a word $w \in \Sigma^*$ if the run of M on w ends in a configuration with the state q_{acc} . Decision TM M rejects a word $w \in \Sigma^*$ if the run of M on w ends in a configuration with the state q_{rej} .

Notice that if the machine does not halt, the run is infinite, and the word is not accepted and not rejected.

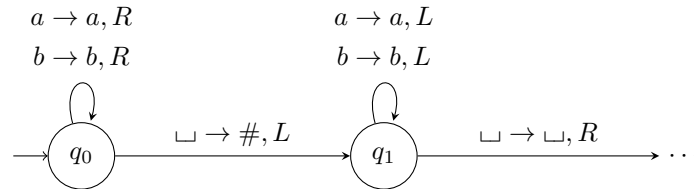
Definition 5 (Language). Given a decision TM M , the language of M is:

$$L(M) := \{w \in \Sigma^* : M \text{ accepts } w\}$$

4 Useful procedures

4.1 Mark with

Suppose we want to mark the end of the input with a special symbol $\#$. This can be useful for example when a TM uses the tape cells to the right of the input to store auxiliary data in its computation, and wants to separate the data from the input. The following TM (with the alphabet $\Sigma = \{a, b\}$) puts $\#$ at the end, and then returns to the beginning of the input.



4.2 Shift right

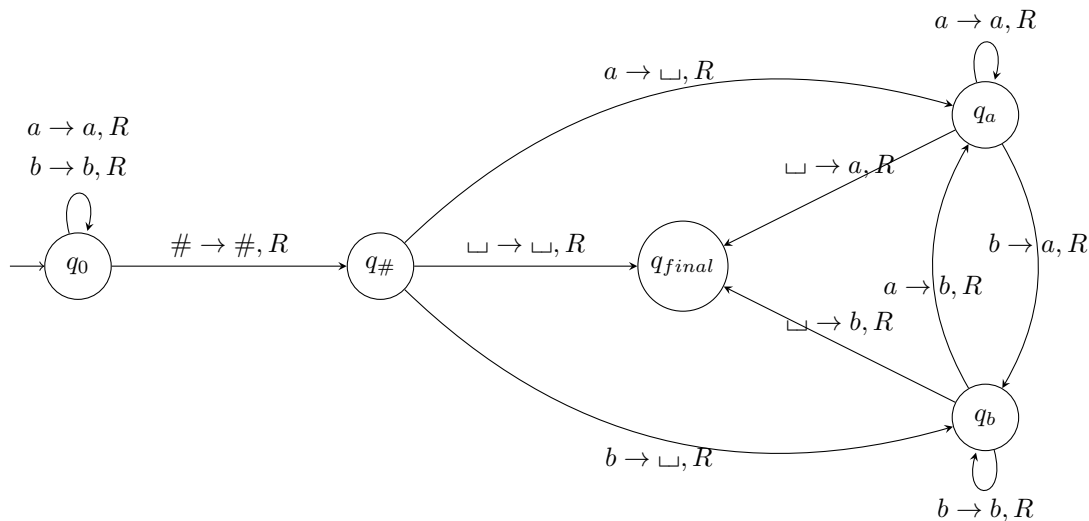
Suppose we want to shift a word one cell to the right on the tape. That is, we want a Turing machine that starts with the input of the form $x\#y$ and ends with $x\#\sqcup y$ written on the tape.

The TM operates as follows:

1. Move the head to the beginning of y , i.e., the cell immediately after the $\#$ symbol.
2. Replace the first symbol of y with a blank $\sqcup$, move the head one cell to the right, and enter a state that remembers the symbol that was overwritten.
3. In each step:
 - Write the symbol remembered from the previous step.
 - Move one cell to the right.
 - Enter a new state that remembers the symbol just overwritten.
4. When a blank symbol is encountered (indicating the end of the word), terminate in a final state.

This procedure shifts the word y one cell to the right, as required.

Let's explicitly construct the TM for the case of $\Sigma = \{a, b\}$:



5 Designing Turing machines

High-level descriptions: After convincing ourselves that TMs can separate parts of the tape, do basic arithmetic, and other “housekeeping” jobs of memory management, we allow slightly higher-level description of algorithms.

5.1 $a^n b^n$

We describe a TM that decides $L = \{a^n b^n : n \geq 0\}$. The idea is to delete a from the beginning and b from the end, repeatedly, until the input is either cleared (and we accept) or the word is not of the form $a^n b^n$ (and we reject).

1. Put two $\#$ symbols, before and after the input.
2. Go to the first non-blank symbol after the left $\#$.
 - (a) If it is $\#$, accept.
 - (b) If it is b , reject.
 - (c) If it is a , remove it (put $\sqcup$ instead) and continue to the next step.
3. Go to the first non-blank symbol before the right $\#$.
 - (a) If it is $\#$, reject.
 - (b) If it is a , reject.
 - (c) If it is b , remove it and go back to step (2).

As we have seen in the above examples, each of those steps can be implemented with appropriate states and transitions.

Remark: Another way to decide L is to use counters. We first make sure that all b 's appear later than all a 's (equivalently, the input should not contain ba). Then we count the occurrences of each letter and compare them. We can count the occurrences of a letter by repeatedly removing them and incrementing a counter.

5.2 Substring

Describe a Turing machine that decides whether a short string is a substring of a longer string. Formally, suppose the input is of the form $x\#y$; the machine should accept if x is a substring of y , and reject otherwise.

We will use a technique where we “write” x above y , allowing us to compare the two in parallel.

We define the tape alphabet to include pairs of symbols stacked vertically. Formally, we define:

$$\Gamma = \Sigma \cup (\Sigma \cup \{\sqcup\}) \times (\Sigma \cup \{\sqcup\})$$

A symbol of the form $\begin{pmatrix} \sigma \\ \tau \end{pmatrix}$ represents writing σ above τ on the same tape cell.

The TM operates as follows:

1. For each symbol τ in y , replace it with $\begin{pmatrix} \sqcup \\ \tau \end{pmatrix}$.
2. Return to the beginning of x (i.e., the start of the tape).
3. For each symbol σ in x , do the following:
 - (a) Replace σ with $\sqcup$.
 - (b) Enter a state that remembers σ .
 - (c) Move right until reaching a cell containing $\begin{pmatrix} \sqcup \\ \tau \end{pmatrix}$ (If you reach $\sqcup$ - reject).
 - (d) Replace it with $\begin{pmatrix} \sigma \\ \tau \end{pmatrix}$.
 - (e) Return to the beginning of (what remains of) x .
4. Move right until reaching the first cell of the form $\begin{pmatrix} \sqcup \\ \tau \end{pmatrix}$ (i.e., the ending of x written above y).
5. Compare x and y symbol by symbol as follows:
 - (a) If the current cell is $\begin{pmatrix} \sigma \\ \sigma \end{pmatrix}$, continue left.
 - (b) If the current cell is $\begin{pmatrix} \sigma \\ \tau \end{pmatrix}$ with $\sigma \neq \tau$, move to a special "mismatch" state and continue left.
6. If you reach a $\begin{pmatrix} \sqcup \\ \tau \end{pmatrix}$ cell while still in the "match" state, accept.
7. Otherwise (i.e., if a mismatch occurred), shift the string x one cell to the right using the algorithm from the previous example. Be sure to keep the lower symbols (those of y) unchanged.
8. Return to step 5 and repeat the comparison at the new position.
9. If, when shifting x to the right, you reach $\sqcup$, reject.

The running time of this algorithm is $O(|x| \cdot |y|)$. This is because we perform up to $|y|$ comparison attempts (sliding x across y), and each comparison takes $O(|x|)$ time. In addition, shifting the string x one cell to the right also takes $O(|x|)$ time. Therefore, the total running time is $O(|x| \cdot |y|)$.

5.3 If time permits - unary to binary

Describe a Turing machine that receives as input a word of the form a^n and outputs the binary representation of n . In the homework, you will implement an algorithm that runs in time $O(n \log n)$. Here, we present a method that runs in $O(n^2)$ time.

The TM operates as follows:

1. Write on the tape the string $\mathbf{x\#0}$.
2. Move the head to the beginning of the tape.
3. Repeat the following:
 - (a) Traverse the tape to the right until reaching the $\#$ symbol.
 - (b) If an unmarked $\mathbf{a}$ is encountered along the way:
 - Mark it (e.g., replace it with $\underline{\mathbf{a}}$).
 - Increment the binary counter (the part after the $\#$) by 1.
 - Return to the beginning of the tape.
 - (c) If no unmarked $\mathbf{a}$ is found:

- Return to the beginning of the tape, converting all marked **a**'s (underscores) back to **a**.
- Halt.

We claim that the running time is $O(n^2)$. This is because there are n iterations (one for each **a**), and in each iteration, we traverse the entire tape (which takes $O(n)$ time), and incrementing the binary counter also takes $O(n)$ time in the worst case.

Therefore, the total time complexity is $O(n^2)$.